

Mattia Robuschi Caprara

Basi di Dati



Prefazione

!!ATTENZIONE!!

Questo documento è un semplice OCR delle slide del prof.

Scopo (pratico) del corso

Nel corso vedremo (parzialmente in parallelo):

1. Come progettare una base di dati sulla base delle specifiche che vengono fornite.
(progettazione concettuale progettazione logica normalizzazione)
2. Come gestire e interrogare la base di dati, una volta che è stata realizzata e "popolata".
(modello relazionale, linguaggio SQL)
3. Come gestire gli accessi concorrenti alla base di dati da parte di più utenti, ma in modo sicuro (nel senso della correttezza del risultato) (transazioni)
4. Esempi di DB non relazionali (NoSQL)

Contents

1	Basi di Dati	4
2	Architettura Client-Server	4
3	DBMS	4
3.1	Affidabilità	4
3.2	Privatezza dei dati	5
3.3	Efficienza	5
3.4	Efficacia	5
3.5	Indipendenza dei dati	5
4	Modelli dei Dati	5
4.1	Il problema fondamentale	6
4.2	Modelli logici dei dati	6
4.3	Modello relazionale	7
4.4	Schema	7
5	Progettazione di una base di dati	8
6	Interfacciarsi con una base di dati	8
6.1	Linguaggi per basi di dati	8
6.1.1	SQL: un linguaggio interattivo	8
6.1.1.1	SQL embedded in Pascal code	9
6.1.1.2	SQL embedded in PHP code	9
6.2	Interazione non testuale	9
6.2.1	Vantaggi dei DBMS	9
6.2.2	Vantaggi dei DBMS	10
7	Modello relazionale	10
7.1	Tre accezioni importanti	10
7.2	Relazione - Prodotto Cartesiano	10
7.2.1	Esempio	10
7.3	Relazioni	10
7.3.1	Esempio	11
7.4	Tupla	11
7.4.1	Perché una tupla si chiama così?	11
7.4.2	Esempio	11
7.5	Cosa significa relazione	11
7.5.1	Proprietà fondamentali	11

7.5.2	Conseguenze	11
7.6	Basi di Dati e Relazioni	11
7.6.1	Esempio	12
7.6.2	Definizione relazione	12
7.6.3	Esempio di relazione con attributi	12
7.6.4	Notazione	12
7.6.5	Esempio	12
7.7	Dominio	12
7.7.1	Esempio	13
7.7.2	Vantaggi dell'approccio basato su valori	13
7.8	Informazione Sbagliata	14
7.9	Vincoli di Integrità	14
7.9.1	Vincoli di Tupla	15
7.9.2	Identificazione delle tuple	15
7.10	Chiavi	15
7.10.1	Esempio	16
7.10.2	Chiave primaria	16
7.10.3	Vincoli di Integrità Referenziale	16
7.10.3.1	Esempio	16
8	Algebra Relazionale	16
8.1	Operatori derivati dagli insiemi	16
8.1.1	Unione	17
8.1.2	Intersezione	17
8.1.3	Differenza	17
8.1.4	Ridenominazione	17
8.2	Selezione e Proiezione	18
8.2.1	Selezione	18
8.2.1.1	Sintassi	19
8.2.1.2	Semantica	19
8.2.1.3	Esempio	19
8.2.1.4	Selezione con valori nulli	19
8.2.2	Proiezione	20
8.2.2.1	Operatore Monadico	20
8.2.2.2	Sintassi	20
8.2.2.3	Semantica	20
8.2.2.4	Esempio	20
8.2.3	Osservazione Importante	21
8.3	Join (naturale)	21
8.3.1	Teorema 1	21
8.3.1.1	Dimostrazione	21
8.3.2	Teorema 2	21
8.3.2.1	Dimostrazione	22
8.3.3	Esempio join completo	22
8.3.4	Esempio join non completo	22
8.3.5	Esempio join vuoto	22
8.3.6	Proprietà	22
8.3.6.1	Theta-join e Equi-join	23
8.3.6.2	Theta-join	23
8.3.6.3	Equi-join	23
8.3.7	Join Esterni	24
8.3.7.1	Join Sinistro	24
8.3.7.2	Join Destro	24
8.3.7.3	Join Completo o Full-Join	24
8.3.8	Join e Proiezioni	25
8.3.8.1	Problema: Perdite	25
8.3.8.2	Problema: Non Reversibilità	25
8.3.8.3	Proprietà	25

9	Interrogazioni (Query)	26
9.1	Esempio	26
9.2	Procedimento Logico	26
9.3	Algebra con valori nulli	27
9.4	Equivalenze di espressioni	27
9.5	Viste (Relazioni Derivate)	28
9.5.1	Vantaggi	28
9.5.2	Esempio	28
9.5.3	Interrogazioni sulle viste	28
9.5.4	Selezione con condizione valutata su tuple diverse della stessa relazione	28
9.5.5	Viste come strumenti di programmazione	29
9.5.6	Viste ed Aggiornamenti	29
10	Esercizi	29

1 Basi di Dati

Informazione: notizia, dato o elemento che consente di avere conoscenza più o meno esatta di fatti, situazioni, modi di essere

Dato: ciò che è immediatamente presente alla conoscenza, prima di ogni elaborazione; (in informatica) elemento di informazione costituito da simboli che devono essere elaborati (dal Vocabolario della Lingua Italiana, Istituto dell'Enciclopedia Italiana)

Base di Dati: collezione di dati, utilizzati per rappresentare le informazioni di interesse per un sistema informativo

2 Architettura Client-Server



Figure 1: Caption

Architettura client-server (applicazione lato server)
DBMS (MySQL/MariaDB), Server Web (APACHE) e interprete PHP sono integrati nel pacchetto XAMPP, scaricabile da questo link
L'applicazione risiede sulla macchina dell'utente e può essere scritta in qualsiasi linguaggio dotato di una API per connettersi, come nel caso di un browser, a un server web (protocollo HTTP) e/o a un server SQL.

3 DBMS

Un Database Management System (DBMS) è un sistema software che si interpone fra le applicazioni e la memoria di massa in cui si trovano collezioni di dati.

La finalità di un DBMS è l'estensione delle funzionalità del file system, in modo da offrire:

- Nuove modalità di accesso ai dati
- Condivisione dei dati
- Gestione più sofisticata dei file

Le basi di dati gestite dai DBMS sono collezioni di dati:

- **Grandi** possono avere notevoli dimensioni (fino a centinaia di Terabyte) e devono quindi necessariamente risiedere nella memoria secondaria
- **Condivise** applicazioni ed utenti diversi devono potere accedere ai dati
- **Persistenti** Il tempo di vita dei dati va oltre la durata dell'esecuzione delle singole applicazioni

Un DBMS deve garantire:

- Affidabilità
- Privacy dei dati
- Efficienza
- Efficacia

3.1 Affidabilità

Un DBMS deve garantire di poter mantenere intatto il suo contenuto, anche in caso di malfunzionamento. L'integrità dei dati è affidata a procedure di backup (salvataggio) e recovery (recupero) dei dati, o alla loro duplicazione nei casi più critici.

3.2 Privatezza dei dati

Ogni utente, abilitato a utilizzare la base di dati attraverso una procedura di riconoscimento, può accedere ad insiemi limitati di dati e compiere solo certe operazioni su di essi.

3.3 Efficienza

Un DBMS deve operare e fornire risposte agli utenti in tempi accettabili, utilizzando una quantità il più possibile limitata di risorse.

L'efficienza dipende essenzialmente dalle tecniche utilizzate per l'implementazione del DBMS e dalla buona progettazione della base di dati.

Si misura (come in tutti i sistemi informatici) in termini di tempo di esecuzione (tempo di risposta) e spazio di memoria (principale e secondaria) occupato.

3.4 Efficacia

Capacità di un DBMS di rendere produttive le attività degli utenti, cioè di consentire la realizzazione di basi di dati che risolvano in modo efficace i problemi degli utenti.

Concetto generico, qualitativo e non legato a specifiche funzionalità del DBMS.

Non esistono criteri oggettivi per valutarla.

3.5 Indipendenza dei dati

Il nuovo strato che il DBMS viene a creare fra memoria di massa e applicazioni consente di conservare e gestire i dati in modo indipendente dalle applicazioni stesse.

Normalmente le applicazioni accedono a dati locali gestendoli attraverso file che appartengono alle applicazioni stesse.

In presenza di un DBMS, i dati non appartengono ad una specifica applicazione, ma le diverse applicazioni vi accedono attraverso di esso.

4 Modelli dei Dati

La progettazione di una base di dati si svolge attraverso una serie di fasi successive, identificabili come:

- Raccolta delle specifiche (identificazione dei concetti)
- Progettazione concettuale
- Progettazione logica
- Progettazione fisica (non compresa nel corso)

Ogni fase fa riferimento ad un modello dei dati che corrisponde ad un meccanismo di strutturazione dei dati attuato a soddisfare le finalità della corrispondente fase della progettazione.

Un modello di dati è costituito dai concetti sulla base dei quali i dati sono strutturati e codificati.

Ogni modello di dati fornisce meccanismi di strutturazione dati, analoghi ai costruttori di tipo dei linguaggi di programmazione.

I modelli concettuali descrivono la realtà mediante concetti astratti, ma soggetti a precise regole.

Non sono finalizzati alla rappresentazione dei dati nel calcolatore, ma a descrivere (e caratterizzare per ruolo) i concetti del mondo reale di cui i dati sono istanze.

Si usano nella fase di progettazione concettuale.

Nel corso useremo il modello Entità/Relazione.



Figure 2: Modello (concettuale) entità-relazione

Si esprimono i concetti che devono essere rappresentati dai dati come concetti indipendenti (entità) e relazioni

logiche che li collegano (relazioni) e che esistono solo in presenza delle entità.

I DBMS si differenziano in base al modello logico che utilizzano.

Quindi i modelli logici, seppure astratti come i modelli concettuali, non hanno solo finalità descrittive e di documentazione, ma riflettono la struttura con cui i dati sono organizzati all'interno del calcolatore.

4.1 Il problema fondamentale

Dato un frammento della realtà che può essere identificato e isolato rispetto al "resto del mondo":

- Descriverlo nel modo più completo possibile, identificando tutti gli elementi (concetti) che lo compongono (e che permettono di distinguerlo da tutto il resto)
- Descrivere, a sua volta, ogni elemento nel modo più corretto possibile, identificandone tutte le caratteristiche (attributi) che permettono di caratterizzarlo nel contesto considerato e ignorando quelle superflue rispetto a tale contesto

Se abbiamo identificato più concetti, questi dovranno risultare logicamente collegati fra loro: ogni concetto deve essere collegato ad almeno un altro concetto, in modo che non vi sia alcun concetto (o insieme di concetti) "isolato" dagli altri.

Se così fosse, infatti, questo costituirebbe un secondo frammento della realtà indipendente da quello considerato e non avrebbe senso descriverli insieme.

Fra i concetti, alcuni (entità) potranno essere descritti in modo indipendente dagli altri attraverso un insieme di attributi che gli sono propri e tipici, senza bisogno di fare riferimento ad altri concetti.

Ad esempio, se il contesto è la gestione di un negozio, posso descrivere un prodotto attraverso il nome, il peso, il prezzo, ecc... e posso descrivere un cliente attraverso i suoi dati anagrafici.

Però non basta, due concetti che posso descrivere in modo indipendente l'uno dall'altro come cliente e prodotto non possono costituire la descrizione completa di un contesto comune.

Per descrivere il contesto del negozio, quindi, cliente e prodotto non possono fare parte della stessa descrizione (schema) senza che esista (almeno) un altro concetto che li colleghi.

D'altra parte, ovviamente, un tale concetto non può, in alcun caso, essere indipendente e non può essere descritto senza fare riferimento ai concetti che collega.

Un concetto che collega altri concetti, e che quindi non può esistere se non esistono i concetti che collega, si chiama relazione o associazione.

Nel caso del negozio, dato il concetto di cliente e di prodotto, una possibile associazione sufficiente a completare lo schema è VENDITA

Ancora non basta, una relazione è un concetto che non è descrivibile soltanto attraverso i riferimenti agli altri concetti (anche più di due) che collega, ma è anch'essa caratterizzata da specifici attributi.

Ad esempio, la descrizione di una vendita dovrà necessariamente contenere l'informazione relativa al cliente cui viene venduto un prodotto e al prodotto che ne è oggetto, ma potrà (e dovrà) anche specificare:

- Data della vendita
- Quantità del prodotto venduto
- Prezzo unitario del prodotto o prezzo totale
- Numero della fattura in cui viene riportata (in questo caso sarà necessario definire un'entità Fattura a cui fare riferimento, quindi estendere lo schema)

4.2 Modelli logici dei dati

- **Relazionale:** Il più diffuso, basato su un modello tabellare dei dati
- **Gerarchico:** Utilizzato nei primi DBMS (anni 60), tuttora utilizzato, basato su strutture ad albero
- **Reticolare:** Estensione del modello gerarchico, basato su grafi
- **A oggetti:** Estensione del modello relazionale basato sui paradigmi OOP.
- **XML (semistrutturato):** Deriva dal modello gerarchico, ma è più flessibile

4.3 Modello relazionale

E' un modello logico: definisce tipi attraverso il costruttore relazione, che organizza i dati secondo record a struttura fissa, rappresentabili attraverso tabelle.

Es. (relazioni INSEGNAMENTO e MANIFESTO)

Corso	Titolare
Basi di Dati e Web	Cagnoni
Fondamenti di Informatica	Tomaiuolo
Ingegneria Del Software	Poggi

CdL	Materia	Anno
IET-I	Basi di Dati e Web	3
IET	Fondamenti di Informatica	1
IET-I	Ingegneria del Software	3

4.4 Schema

In ogni base di dati si possono distinguere:

- **Lo schema**, sostanzialmente invariante nel tempo. Ne descrive la struttura (si parla di aspetto intensionale).
Nell'esempio, le tabelle e le "intestazioni" delle tabelle
Analogo della definizione di una classe in un linguaggio a oggetti
- **Le istanze**, cioè i valori attuali, che possono cambiare anche molto rapidamente (si parla di aspetto estensionale).
Nell'esempio, i singoli dati contenuti in ciascuna tabella
A livello concettuale, analogo di un oggetto, come istanza di una classe, in un linguaggio a oggetti

Lo schema di una base di dati è la parte dichiarativa ed invariante della base di dati e ne definisce la struttura. Nel modello relazionale lo schema di una relazione è paragonabile alla definizione del prototipo di una funzione in C:

INSEGNAMENTO (Corso, Titolare)

è lo schema della relazione INSEGNAMENTO definita su due attributi (campi del record identificati da una etichetta che ne descrive il significato).

Le n-pie di attributi appartenenti alla relazione, a ciascuno dei quali deve essere assegnato un valore, sono dette istanze della relazione.

(Basi di dati, Cagnoni) è una istanza di INSEGNAMENTO

Gli schemi possono operare a diversi livelli di astrazione.

Dalle specifiche è possibile derivare lo **Schema concettuale** che descrive i concetti che devono essere rappresentati, i dettagli che servono per descriverli e le relazioni logiche che li collegano.

Dallo schema concettuale si deriva lo **Schema logico** che descrive la struttura dell'intera base di dati mediante il modello logico adottato dal DBMS (reticolare, gerarchico, relazionale).

Che viene poi realizzato attraverso lo **Schema interno (o schema fisico)** che implementa lo schema logico mediante strutture fisiche di memorizzazione

Nei confronti dell'utente è possibile definire anche uno **Schema esterno** che descrive la struttura di una porzione della base di dati attraverso il modello logico, dal punto di vista di una classe di utenti.

Generalmente realizzato mediante viste, relazioni derivate da quelle che costituiscono lo schema logico.

Es. (i soli corsi del curriculum Ingegneria Informatica):

CdL	Materia	Anno
IET-I	Basi di Dati e Web	3
IET-I	Ingegneria del Software	3



Figure 3: Architettura standard (ANSI/SPARC) a tre livelli per DBMS

5 Progettazione di una base di dati

I diversi tipi di schema riflettono le diverse fasi della progettazione: dalle specifiche del problema da risolvere

- **Progettazione concettuale** (dalle specifiche allo schema concettuale)
Definisce quali concetti rappresentare, per quanto riguarda sia la loro descrizione che le relazioni logiche che esistono fra di essi.
- **Progettazione logica** (dallo schema concettuale allo schema logico)
Definisce come rappresentare i concetti introdotti a livello concettuale in forma di strutture dati.
- **Progettazione fisica** (dallo schema logico allo schema fisico)
Definisce come allocare e gestire fisicamente i dati all'interno del calcolatore.

6 Interfacciarsi con una base di dati

Si può eseguire l'accesso ad una base di dati nei seguenti modi:

- Linguaggi testuali interattivi
- Comandi inclusi in estensioni di linguaggi tradizionali
- Comandi inclusi in linguaggi di sviluppo ad hoc
- Interfacce grafiche amichevoli

6.1 Linguaggi per basi di dati

- Linguaggi di definizione dei dati
Utilizzati per definire gli schemi e le autorizzazioni per l'accesso
- Linguaggi di manipolazione dei dati
Utilizzati per l'interrogazione e l'aggiornamento dei contenuti della base di dati

Alcuni linguaggi specializzati, come ad esempio SQL, presentano le caratteristiche di entrambi i tipi di linguaggio.

6.1.1 SQL: un linguaggio interattivo

Come esempio immaginiamo di interfacciarci con la seguente base di dati:

Corsi	
Corso	Aula
Sistemi	N3
Reti	N2

Aule	
Nome	Piano
N3	Terra
N2	Terra

Corso	Aula	Piano
Sistemi	N3	Terra
Reti	N2	Terra

```

1 SELECT Corso, Aula, Piano
2 FROM Aule, Corsi
3 WHERE Nome = Aula AND Piano = "Terra"
4
```

6.1.1.1 SQL embedded in Pascal code

Di seguito un esempio di SQL integrato in un linguaggio di alto livello:

```
1  write('nome della citta''?');
2  readln(citta);
3  EXEC SQL DECLARE P CURSOR FOR
4  SELECT NOME, REDDITO
5  FROM PERSONE
6  WHERE CITTA = :citta ;
7  EXEC SQL OPEN P ;
8  EXEC SQL FETCH P INTO :nome, :reddito ;
9  while SQLCODE = 0 do begin
10 write('nome della persona:', nome, 'aumento?');
11 readln(aumento);
12 EXEC SQL UPDATE PERSONE SET REDDITO = REDDITO +
13 :aumento
14 WHERE CURRENT OF P
15 EXEC SQL FETCH P INTO :nome, :reddito
16 end;
17 EXEC SQL CLOSE CURSOR P
18
```

6.1.1.2 SQP embedded in PHP code

Connessione a mySQL tramite API PHP:

```
1  <html> <body>
2  <?php
3  $user="root"; $password=""; $host="127.0.0.1"; $database="prova";
4  mysql_connect($host,$user,$password);
5  @mysql_select_db($database) or die("Unable to select database");
6  $query="SELECT * from utenti";
7  $res=mysql_query($query);
8  mysql_close();
9
10 $row = mysql_fetch_assoc($res);
11 print '<table><tr>';
12 foreach($row as $name => $value) {
13     print "<th>$name</th>";
14 }
15 print '</tr>';
16
17 while($row) {
18     print '<tr>';
19     foreach($row as $value) {
20         print "<td>$value</td>";
21     }
22     print '</tr>';
23     $row = mysql_fetch_assoc($res);
24 }
25 print '</table>';
26 ?> </body> </html>
27
```

6.2 Interazione non testuale

Come interazioni non testuali si intendono generalmente i DBMS come ad esempio Microsoft Access:



Figure 4: Realizza l'approccio detto "Query by Example"

6.2.1 Vantaggi dei DBMS

- Disponibilità dei dati a tutta una comunità

- Modello unificato e preciso della realtà di interesse
- Controllo centralizzato dei dati
- Condivisione
- Indipendenza dei dati

6.2.2 Vantaggi dei DBMS

- Prodotti costosi, complessi, che richiedono investimenti in hardware, software, personale.
- Forniscono un numero elevato di servizi, in modo integrato e difficilmente scorporabile se le esigenze dell'utente sono inferiori alle caratteristiche offerte.

7 Modello relazionale

- Proposto agli inizi degli anni '70 da Codd
- Finalizzato alla realizzazione dell'indipendenza dei dati
- Unisce concetti derivati dalla teoria degli insiemi (concetto di relazione) con una rappresentazione dei dati di tipo tabellare
- Attualmente è largamente il modello più utilizzato
- Teorizzato per separare il più possibile il livello logico dal livello fisico della descrizione dei dati: la base di dati non contiene alcun riferimento all'effettiva allocazione dei dati in memoria.
- Basato su un rigoroso modello matematico, permette un elevato grado di astrazione.
- Rappresentazione tabellare: semplice ed intuitiva.
Le relazioni ed i risultati delle operazioni sulle tabelle sono facilmente rappresentabili ed interpretabili dagli utenti.

7.1 Tre accezioni importanti

1. Relazione matematica: come nella teoria degli insiemi
2. Relazione secondo il modello relazionale dei dati
3. Relazione (dall'inglese **relationship**) di tipo **logico** fra entità (concetti descrivibili in modo indipendente dagli altri) nel modello Entità-Relazione (entity-relationship); traducibile anche con **associazione** o **correlazione** (scelta preferibile per evitare ambiguità).

7.2 Relazione - Prodotto Cartesiano

Dati due insiemi D_1 e D_2 si definisce **Prodotto Cartesiano** di D_1 e D_2 ($D_1 \times D_2$) l'insieme di tutte le possibili coppie ordinate (v_1, v_2) tali che v_1 sia un elemento di D_1 e v_2 sia un elemento di D_2 .

7.2.1 Esempio

Dati gli insiemi: $A = \{cubo, cono\}$ e $B = \{rosso, verde, blu\}$

Il prodotto cartesiano tra A e B è $\{(cubo, rosso), (cono, rosso), (cubo, verde), (cono, verde), (cubo, blu), (cono, blu)\}$

7.3 Relazioni

Una relazione matematica su due insiemi D_1 e D_2 è un sottoinsieme del prodotto cartesiano $D_1 \times D_2$.

NOTA: a livello formale gli insiemi possono essere infiniti, a livello pratico non possiamo però considerare relazioni infinite.

7.3.1 Esempio

Dati gli insiemi visti, una possibile relazione è $\{(\text{cubo}, \text{rosso}), (\text{cono}, \text{rosso}), (\text{cubo}, \text{blu})\}$

o, in forma tabellare:

Cubo	Rosso
Cono	Rosso
Cubo	Blu

7.4 Tupla

Le definizioni viste per due insiemi possono essere generalizzate a n insiemi.

Ogni riga della tabella sarà allora una n -pla ordinata di elementi (detta tupla).

n è detto **grado** del prodotto cartesiano e quindi della relazione.

Il numero di elementi (istanze) della relazione è detto **cardinalità** della relazione.

Un insieme può apparire più volte in una relazione.

7.4.1 Perché una tupla si chiama così?

Perché è una n -upla (ennupla) che viene rappresentata di solito con la lettera t anziché con la lettera n

7.4.2 Esempio

La relazione `Partite_di_Calcio(Casa, Ospiti, RetiCasa, RetiOspiti)` è un sottoinsieme del prodotto cartesiano **Stringa x Stringa x Intero x Intero**.

7.5 Cosa significa relazione

Una relazione è concetto mutuato dalla definizione di relazione matematica della teoria degli insiemi, definito come sottoinsieme del prodotto cartesiano fra n insiemi.

Nel modello relazionale corrisponde ad una struttura dati tabellare.

7.5.1 Proprietà fondamentali

- Ogni tupla è internamente ordinata: l' i -esimo valore proviene dall' i -esimo dominio (struttura posizionale)
- Non esiste ordinamento intrinseco fra le tuple, per la natura insiemistica della relazione
- Non sono ammesse tuple uguali (ogni elemento di un insieme è unico)

7.5.2 Conseguenze

- Lo scambio fra righe di una tabella non modifica la relazione
- Lo scambio fra colonne di una tabella può portare alla sua inconsistenza con lo schema

7.6 Basi di Dati e Relazioni

Uno **schema di relazione** $R(X)$ è costituito da un simbolo (nome della relazione) R e da una serie di attributi $X = A_1, A_2, \dots, A_n$

Quindi una tupla t è una istanza di una relazione r (un elemento del corrispondente insieme) la cui struttura è definita dal corrispondente schema $R(X)$.

Se la relazione è rappresentata come tabella, una istanza della relazione è una riga della tabella:

01	Analisi	Rossi
----	---------	-------

Uno **schema di base di dati** è un insieme di schemi di relazione con nomi diversi

$$R = R_1(X_1), R_2(X_2), \dots, R_n(X_n)$$

Una **base di dati** definita su uno schema

$$R = R_1(X_1), R_2(X_2), \dots, R_n(X_n)$$

è un insieme di relazioni $r = r_1, r_2, \dots, r_n$ dove ogni r_i è una relazione sullo schema $R_i(X_i)$.

A livello di rappresentazione, una base di dati è un **insieme di tabelle**.

7.6.1 Esempio

Corsi(*Codice, Titolo, Docente*)

Una relazione su uno schema $R(X)$ è un insieme r di tuple definite su X : è rappresentata come una tabella:

Corsi		
Codice	Titolo	Utente
01	Analisi	Rossi
02	Chimica	Bruni
04	Chimica	Verdi

7.6.2 Definizione relazione

Una relazione è un **insieme di record omogenei**, cioè definiti sugli stessi campi.

Come ogni campo di un record è associato ad un **nome (etichetta)**, così si associa ad ogni colonna della relazione un **attributo**.

7.6.3 Esempio di relazione con attributi

Corsi			
Casa	Ospiti	RetiCasa	RetiOspiti
Parma	Inter	3	2
Palermo	Lazio	2	0
Milan	Juventus	1	1

7.6.4 Notazione

Se t è una tupla definita sullo schema $R(X)$ (insieme ordinato di domini) di una relazione e A è uno dei domini di $R(X)$.

$t[A]$ (o $t.A$) è il valore di t relativo al dominio A .

7.6.5 Esempio

[relazione $Partite(Casa, Ospiti, RetiCasa, RetiOspiti)$]

Se t è la prima tupla della relazione (vedi pagine precedenti), allora $t.Casa = Parma$



Figure 5: Livello descrittivo e Rappresentazione

7.7 Dominio

Ogni attributo ha un suo **dominio** su cui è definito.

Quindi una tupla è un insieme di valori, uno per attributo, ordinati secondo lo schema della relazione e definiti ciascuno su un proprio dominio.

Una relazione è una serie di tuple definite sullo schema della relazione (insieme ordinato dei domini dei singoli attributi).

Studenti(<i>Matricola, Cognome, Nome, DataNascita</i>)
Corsi(<i>Codice, Titolo, Docente</i>)
Esami(<i>Studente, Voto, Corso</i>)

- **Studenti** contiene dati su un insieme di studenti.
- **Corsi** contiene dati su un insieme di corsi.
- **Esami** contiene dati su un insieme di esami e fa riferimento alle altre due attraverso i numeri di matricola e il codice del corso.



Figure 6: Schema logico base di dati

Quindi Matricola e Studente, come Corso e Codice, sono definiti sullo stesso dominio e possono (in questo caso devono, per generare il riferimento) assumere gli stessi valori. Lo schema logico della base di dati è la descrizione, a livello logico, di uno schema concettuale in cui, se si usa la rappresentazione col modello Entità/Relazione:

- Studenti e Corsi sono due entità, perché sono concetti che esistono indipendentemente dagli altri e possono essere rappresentati e descritti in modo autonomo.
- Esami è una relazione, perché è un concetto che esiste solo come collegamento (correlazione, associazione) fra due concetti rappresentabili come entità.

Importante notare:

- A livello concettuale, l'unico attributo dell'entità Esami è Voto.
Gli attributi di Esami che servono come riferimenti, cioè Studente (verso la tabella Studenti) e Corso (verso la tabella Corsi), sono attributi di Esami nel modello logico solo per tradurre l'informazione che nel diagramma E/R è fornita dal legame grafico e specifica che Esami è un'associazione che collega le entità Corsi e Studenti.
- Come attributi, Studente e Corso sono definiti su un dominio semplice (cioè ogni attributo ha un solo valore) e quindi fanno riferimento a una sola istanza di Studenti o Corsi.

7.7.1 Esempio

Studenti				Esami			Corsi		
Matricola	Cognome	Nome	Data di nascita	Studente	Voto	Corso	Codice	Titolo	Docente
6554	Rossi	Mario	05/12/1978	3456	30	04	01	Analisi	Mario
8765	Neri	Paolo	03/11/1976	3456	24	01	02	Chimica	Bruni
9283	Verdi	Luisa	12/11/1979	9283	28	01	04	Chimica	Verdi
3456	Rossi	Maria	01/02/1978	6554	26	02			

Dall'esempio abbiamo visto che:



Figure 7: Esplicitazione relazioni fra le tabelle

- Il modello relazionale è basato su valori.
- I riferimenti fra dati in relazioni diverse avvengono attraverso la corrispondenza dei valori con i quali (insiemi di) attributi corrispondenti vengono istanziati nelle tuple che sono logicamente correlate.

NB Attributi corrispondenti può implicare uguaglianza del nome oppure corrispondenza (logica) fra valori ammissibili, attraverso l'imposizione di vincoli. In genere i due (insiemi di) attributi ammettono gli stessi valori oppure, per uno dei due, sono ammissibili i valori di un sottoinsieme dei valori ammissibili per l'altro.

Gli altri modelli (gerarchico, reticolare) utilizzano puntatori per realizzare la corrispondenza.

7.7.2 Vantaggi dell'approccio basato su valori

- Si inseriscono nella base di dati solo valori significativi per l'applicazione (i puntatori sono dati aggiuntivi relativi alla sola implementazione fisica).

- Il trasferimento dei dati da un ambiente ad un altro è più semplice (i puntatori hanno validità solo locale)
- La rappresentazione logica dei dati non fa riferimento a quella fisica e quindi si ottiene l'indipendenza (fisica) dei dati

7.8 Informazione Sbagliata

Le tuple che compongono la base di dati devono essere omogenee.
Quindi, in ogni tupla, ad ogni attributo deve essere associato un valore.
Non sempre questo è possibile.

Es. Persone(*Cognome, Nome, Indirizzo, Telefono*)

Potrebbe esistere una persona che non possiede telefono, o di cui non conosciamo l'indirizzo.

Cognome	Nome	Indirizzo	Telefono
Rossi	Francesco	v. Rossa 22	0521 335643
Verdi	Giuseppe	v. Verde 11	
Bruni	Bruno		336 7564213

Questa relazione non è ammissibile! In ogni tupla, ogni attributo deve essere istanziato.
Non conviene (anche se è un espediente di uso comune) usare valori del dominio normalmente non utilizzati (0, stringa nulla, "99", ...), come spesso accade nella programmazione:

- Potrebbero non esistere valori non utilizzati
- Valori non utilizzati potrebbero, a un certo punto, diventare significativi ed essere utilizzati
- In fase di utilizzo (nei programmi) sarebbe necessario ogni volta tenere conto del "significato" (non standardizzato) di questi valori

Nel modello relazionale è definito un valore convenzionale, detto valore nullo, che indica la non disponibilità dell'informazione.

Il valore nullo può rappresentare correttamente tre situazioni logicamente diverse in cui l'informazione è:

- Sconosciuta (so che il valore esiste ma non lo conosco)
- Inesistente (so che il valore non esiste)
- Indeterminata (non so se il valore esiste e, in ogni caso, non lo conosco)

7.9 Vincoli di Integrità

Non tutte le combinazioni possibili di valori dei domini su cui è definita una relazione sono accettabili.

- Alcuni attributi possono assumere valori solo in un certo intervallo
- Alcuni attributi devono essere diversi in ogni tupla della stessa relazione

Es. valori dell'attributo Matricola nella relazione
Studenti(*Matricola, Cognome, Nome, DataNascita*)

Alcuni valori possono essere incompatibili con altri all'interno della stessa relazione

Es. data la relazione
Esami(*Matricola, Voto, Lode, CodCorso*)

1. Una stessa coppia Matricola, Corso può apparire una sola volta
2. Il valore Vero per l'attributo Lode è corretto solo se Voto=30

Alcuni valori possono essere incompatibili con i valori di un'altra relazione

Es. date le relazioni:
Studenti (*Matricola, Cognome, Nome, DataNascita*)
Esami (*Studiante, Voto, Lode, CodCorso*)
Corsi (*CodCorso, Titolo, Docente*)

1. Ogni valore di CodCorso in Esami, in questo caso, DEVE essere un valore esistente di CodCorso nella relazione Corsi;
2. Analogamente, ogni valore dell'attributo Studente nella tabella Esami DEVE essere un valore esistente dell'attributo Matricola nella relazione Studenti.

Sono condizioni, espresse come predicati logici, inserite nella base di dati per garantirne la consistenza.

Ogni istanza della base di dati deve soddisfare i vincoli di integrità, cioè il predicato corrispondente al vincolo deve assumere valore vero **in ogni istante**.

Una istanza che soddisfi tutti i vincoli è detta corretta (o lecita, o ammissibile)

Un vincolo è detto:

- **Intra-relazionale** se coinvolge attributi della stessa relazione

Es.

- **Vincoli di tupla** possono essere valutati su ciascuna tupla indipendentemente dalle altre
- **Vincoli di dominio** definiti su singoli valori
- **Interrelazionale** se coinvolge più relazioni

7.9.1 Vincoli di Tupla

Possono essere definiti attraverso operatori booleani

Es. Data la relazione:

Esami(*Matricola*, *Voto*, *Lode*, *CodCorso*)

```

1  (Voto >= 18) AND (Voto <= 30)
2  (NOT (lode=Vero)) OR (Voto=30)
3

```

Oppure, data la relazione:

Pagamenti(*Data*, *Importo*, *Ritenute*, *Netto*)

```

1  Netto = Importo - Ritenute
2

```

7.9.2 Identificazione delle tuple

Matricola	Cognome	Nome	Corso	Nascita
27655	Rossi	Mario	Ing Inf	5/12/78
78763	Rossi	Mario	Ing Inf	3/11/76
65432	Neri	Piero	Ing Mecc	10/7/79
87654	Neri	Mario	Ing Inf	3/11/76
67653	Rossi	Piero	Ing Mecc	5/12/78

Osservazioni:

- Non ci sono due tuple con lo stesso valore sull'attributo Matricola
- Non ci sono due tuple uguali su tutti e tre gli attributi Cognome, Nome e Nascita

7.10 Chiavi

Una chiave è un insieme minimale di attributi utilizzato per identificare univocamente le tuple di una relazione.

Formalmente: Un insieme di attributi K è superchiave per una relazione r se r non contiene due tuple t_1 e t_2 tali che $t_1[K] = t_2[K]$

Un insieme di attributi K è chiave per r se è superchiave minimale, cioè se non esiste un'altra superchiave K' che sia sottoinsieme di K o, comunque, di grado inferiore.

Una chiave è tale se soddisfa la definizione per tutte le possibili tuple appartenenti alla relazione, e non solo per quelle che effettivamente appaiono come istanze della relazione stessa.

⇒ La chiave è legata allo schema della relazione e non ai valori effettivamente assunti dalle istanze dello schema.

Ogni relazione, per definizione, possiede una chiave.

Infatti, poiché una relazione non ammette due tuple uguali, l'intero insieme di attributi X su cui è definita è sicuramente superchiave per la relazione.

Può possederne anche più di una.

7.10.1 Esempio

Matricola	Cognome	Nome	Corso	Nascita
27655	Rossi	Mario	Ing Inf	5/12/78
78763	Rossi	Mario	Ing Inf	3/11/76
65432	Neri	Piero	Ing Mecc	10/7/79
87654	Neri	Mario	Ing Inf	3/11/76
67653	Rossi	Piero	Ing Mecc	5/12/78

Matricola è una chiave:

- è superchiave
- contiene un solo attributo e quindi è minimale

7.10.2 Chiave primaria

La presenza di valori nulli in una chiave può vanificare la proprietà di unicità delle tuple che identifica. Si impone quindi che almeno una chiave non contenga valori nulli.

Tale chiave è detta chiave primaria.

Di solito la chiave primaria compare sottolineata nello schema di una relazione.

Es.

Studenti(Matricola, *Cognome*, *Nome*, *Nascita*, *Corso*)

7.10.3 Vincoli di Integrità Referenziale

In alcuni casi (corrispondenze fra relazioni) è necessario che i valori degli attributi di una relazione R_1 si trovino anche in attributi corrispondenti di un'altra relazione R_2 .

Un **vincolo di integrità referenziale** (o **foreign key**, o **chiave esterna**) fra un insieme di attributi X di R_1 e un'altra relazione R_2 è soddisfatto se i valori su X di ciascuna tupla di R_1 compaiono come valori della chiave (primaria) di R_2 .

7.10.3.1 Esempio

Corsi (*Corso*, *Ncredits*)

Manifesto (*CdL*, *Insegnamento*, *Anno*)

Esiste un vincolo di integrità referenziale fra Insegnamento, attributo di Manifesto, e Corso, chiave primaria di Corsi

Corso	Ncredits
Basi di Dati e Web	9
Sistemi Operativi	6
Ingegneria Del Software	9

CdL	Insegnamento	Anno
IET-I	Basi di Dati e Web	3
IET	Sistemi Operativi	2
IET-I	Ingegneria del Software	3

8 Algebra Relazionale

Linguaggio procedurale: le operazioni vengono descritte specificando la sequenza di azioni da compiere per ottenere la soluzione.

Operatori:

- **unione**, **intersezione**, **differenza** derivati dalla teoria degli insiemi
- **ridenominazione**, **selezione**, **proiezione** specifici dell'algebra relazionale
- **join** può assumere diverse forme (**naturale**, **theta-join**, **prodotto cartesiano**)

8.1 Operatori derivati dagli insiemi

Usati anche nella teoria degli insiemi: unione, intersezione, differenza.

Le relazioni sono insiemi e quindi è naturale estendere ad esse tali operazioni.

NB. Il risultato di qualunque operazione fra relazioni deve essere ancora una relazione.

Le relazioni sono insiemi di tuple omogenee: un risultato che comprende tuple definite su schemi diversi non è una relazione.

Ha quindi senso definire ed applicare tali operatori solo a tuple definite sugli stessi attributi.

Es.

l'unione fra due relazioni su tuple non omogenee non è una relazione.

- **Unione:**

L'unione fra due relazioni r_1 e r_2 definite sullo stesso schema X si indica con $r_1 \cup r_2$ ed è una relazione su X contenente le tuple appartenenti a r_1 , a r_2 o ad entrambe.

- **Intersezione:**

L'intersezione fra due relazioni r_1 e r_2 definite sullo stesso schema X si indica con $r_1 \cap r_2$ ed è una relazione su X contenente le tuple appartenenti sia a r_1 che a r_2 .

- **Differenza:**

La differenza fra due relazioni r_1 e r_2 definite sullo stesso schema X si indica con $r_1 - r_2$ ed è una relazione su X contenente le tuple appartenenti a r_1 ma non a r_2 .

8.1.1 Unione

Laureati			Quadri		
Matricola	Nome	Età	Matricola	Nome	Età
7274	Rossi	42	9297	Neri	33
7432	Neri	54	7432	Neri	54
9824	Verdi	45	9824	Verdi	45

Laureati $\cup$ Quadri		
Matricola	Nome	Età
7274	Rossi	42
7432	Neri	54
9824	Verdi	45
9297	Neri	33

8.1.2 Intersezione

Laureati			Quadri		
Matricola	Nome	Età	Matricola	Nome	Età
7274	Rossi	42	9297	Neri	33
7432	Neri	54	7432	Neri	54
9824	Verdi	45	9824	Verdi	45

Laureati $\cap$ Quadri		
Matricola	Nome	Età
7432	Neri	54
9824	Verdi	45

8.1.3 Differenza

Laureati			Quadri		
Matricola	Nome	Età	Matricola	Nome	Età
7274	Rossi	42	9297	Neri	33
7432	Neri	54	7432	Neri	54
9824	Verdi	45	9824	Verdi	45

Laureati - Quadri		
Matricola	Nome	Età
7274	Rossi	42

8.1.4 Ridenominazione

In algebra relazionale si ha corrispondenza fra attributi mediante il nome.

La ridenominazione modifica il nome di un attributo (ad es., per associarlo ad un altro in un'operazione algebrica).

Può, ad es., rendere compatibili due attributi con nome diverso quando ha senso eseguire l'unione fra le relazioni

cui appartengono.

Si indica con $\rho_{nuovonome \leftarrow vecchionome}$ (Relazione)

Es.

Da Paternità(*Padre, Figlio*) a Maternità(*Madre, Figlio*) è possibile ottenere:

$$\rho_{Genitore \leftarrow Padre} \text{ (Paternità)} \cup \rho_{genitore \leftarrow Madre} \text{ (Maternità)}$$

REN _{Genitore ← Padre} (Paternità)		REN _{Genitore ← Madre} (Maternità)	
Genitore	Figlio	Genitore	Figlio
Adamo	Abele	Eva	Abele
Adamo	Caino	Eva	Set
Abramo	Isacco	Sara	Isacco

REN _{Genitore ← Padre} (Paternità) $\cup$ REN _{Genitore ← Madre} (Maternità)	
Genitore	Figlio
Adamo	Abele
Adamo	Caino
Abramo	Isacco
Eva	Abele
Eva	Set
Sara	Isacco

8.2 Selezione e Proiezione

Le operazioni di **selezione** e di **proiezione** si applicano ad una relazione e ne restituiscono una porzione. Possono essere considerate **ortogonali** o **complementari**, in quanto una opera sulle righe e l'altra sulle colonne.

La selezione produce un insieme di tuple, definite su tutti gli attributi della relazione.

La proiezione produce un risultato definito su un insieme limitato di attributi, cui contribuiscono tutte le tuple



Figure 8: Selezione & Proiezione

8.2.1 Selezione

La selezione produce una nuova relazione definita sugli stessi attributi della relazione cui è applicata, contenente solamente le tuple di quest'ultima che soddisfano una specifica condizione di selezione.

Si indica con $\sigma_F(r)$ o $SEL_F(r)$ dove:

- F è una condizione da verificare
- r è la relazione a cui la selezione è applicata

Quindi $\sigma_F(r)$ produce una relazione, sullo stesso schema di r , contenente tutte le tuple per le quali F è vera.

La condizione di selezione F è una formula proposizionale su X , cioè una formula ottenuta combinando con i simboli $\wedge$ (and) $\vee$ (or) $\neg$ (not) espressioni del tipo $A\theta B$ o $A\theta c$.

- θ è un operatore di confronto ($\leq, <, =, >, \geq$)
- A e B sono attributi di X su cui il confronto abbia senso
- c è una costante tale che il confronto con A sia definito

È definito un valore di verità di F su una tupla t :

- $A\theta B$ è vera se e solo se $t[A]\theta t[B]$ è vero
- $A\theta c$ è vera se e solo se $t[A]\theta c$ è vera
- $F_1 \wedge F_2, F_1 \vee F_2, \neg F$ hanno l'usuale significato

8.2.1.1 Sintassi

$$SEL_{Condizione}(Operando)$$

Condizione: espressione booleana (come quelle dei vincoli di tupla)

8.2.1.2 Semantica

Il risultato contiene le tuple dell'operando che soddisfano la condizione

8.2.1.3 Esempio

Impiegati			
Matricola	Cognome	Filiale	Stipendio
7309	Rossi	Roma	55
5998	Neri	Milano	64
9553	Rossi	Milano	44
5698	Neri	Napoli	64

$$SEL_{Stipendio > 50}(Impiegati)$$

Impiegati che guadagnano più di 50			
Matricola	Cognome	Filiale	Stipendio
7309	Rossi	Roma	55
5998	Neri	Milano	64
5698	Neri	Napoli	64

8.2.1.4 Selezione con valori nulli

Impiegati			
Matricola	Cognome	Filiale	Età
7309	Rossi	Roma	32
5998	Neri	Milano	45
9553	Bruni	Milano	NULL

$$SEL_{Età > 30}(Impiegati) \cup SEL_{Età \leq 30}(Impiegati) \neq Impiegati$$

(non comprende le tuple con Età=NULL)

Ma anche:

$$SEL_{Età > 30 \vee Età \leq 30}(Impiegati) \neq Impiegati$$

(come sopra)

Per riferirsi ai valori nulli esistono specifiche condizioni logiche:

- x IS NULL (vera se, per una tupla t , $t[x] = \text{NULL}$)
- x IS NOT NULL (vera se, per una tupla t , $t[x] \neq \text{NULL}$)

$$SEL_{Età > 30}(Impiegati) \cup SEL_{Età \leq 30}(Impiegati) \cup SEL_{Età \text{ IS NULL}}(Impiegati)$$

=

$$SEL_{Età > 30 \vee Età \leq 30 \vee Età \text{ IS NULL}}(Impiegati)$$

=

$$Impiegati$$

8.2.2 Proiezione

Dati una relazione $r(X)$ e un sottoinsieme Y di X , la proiezione di r su Y si indica con:

$$\Pi_Y(r) \text{ oppure } PROJ_Y(r)$$

ed è l'insieme di tuple su Y ottenute dalle tuple di r considerando solo i valori su Y :

$$\Pi_Y(r) = \{t[Y] \mid t \in r\}$$

Una proiezione ha un numero di tuple minore o uguale rispetto alla relazione r cui è applicata: limitare il numero di attributi può portare a creare duplicati.

Quindi il numero di tuple è uguale se e solo se Y è superchiave per r .

8.2.2.1 Operatore Monadico

Produce un risultato che

- Possiede parte degli attributi dell'operando
- Contiene tuple cui contribuiscono tutte le tuple dell'operando

8.2.2.2 Sintassi

$$PROJ_{ListaAttributi}(Operando)$$

8.2.2.3 Semantica

Il risultato contiene le tuple che si ottengono restringendo tutte le tuple dell'operando agli attributi nella lista (ed eliminando gli eventuali duplicati)

8.2.2.4 Esempio

Impiegati			
Matricola	Cognome	Filiale	Stipendio
7309	Neri	Napoli	55
5998	Neri	Milano	64
9553	Rossi	Roma	44
5698	Rossi	Roma	64

Visualizzare matricola e cognome di tutti gli impiegati:

Impiegati	
Matricola	Cognome
7309	Neri
5998	Neri
9553	Rossi
5698	Rossi

$$PROJ_{Matricola,Cognome}(Impiegati)$$

Visualizzare cognome e filiale di tutti gli impiegati:

Impiegati	
Cognome	Filiale
Neri	Napoli
Neri	Milano
Rossi	Roma
Rossi	Roma

$$PROJ_{Cognome,Filiale}(Impiegati)$$

8.2.3 Osservazione Importante

Combinando selezione e proiezione, si può estrarre qualunque gruppo di elementi (anche uno solo, ad es. lo stipendio dell'impiegato con matricola 7309) da una singola relazione (nell'esempio, la tabella Impiegati)

Non si possono però correlare informazioni presenti in relazioni diverse

Questa considerazione suggerisce che un modo per estrarre qualsiasi informazione da una base di dati costituita da più relazioni sia:

1. Definire un operatore che consenta di trasformare in un'unica relazione più relazioni contenenti informazioni correlate
2. Usare selezione e proiezione per estrarre i dati dalla relazione risultante

8.3 Join (naturale)

È l'operatore più caratteristico dell'algebra relazionale, che evidenzia la proprietà del modello relazionale di essere basato su valori.

Non ha un corrispettivo nella teoria degli insiemi.

L'operatore di join (naturale) correla dati contenuti in relazioni diverse.

Nella situazione più comune, produce una relazione definita sull'unione degli schemi degli operandi, le cui tuple sono ottenute combinando le tuple degli operandi che hanno valori uguali su attributi comuni (cioè che hanno lo stesso nome nei due schemi).

Il join naturale $r_1 \Join r_2$ di $r_1(X_1)$ e $r_2(X_2)$ è una relazione definita su $X_1 \cup X_2$ (Che si può scrivere come $X_1 X_2$)

$$r_1 \Join r_2 = \{t \text{ su } X_1 X_2 \mid t[X_1] \in r_1 \text{ e } t[X_2] \in r_2\}$$

Il grado di tale relazione (numero attributi) è minore o uguale alla somma dei gradi delle due relazioni poiché gli attributi omonimi compaiono una sola volta.

Per questo motivo la definizione implica, normalmente, che una tupla del risultato sia ottenuta concatenando ogni tupla della prima relazione con ogni tupla della seconda in cui gli attributi comuni hanno lo stesso valore.

Due casi limite:

- Join vuoto: le due tabelle hanno alcuni attributi in comune ma nessuna tupla di un operando è combinabile con nessuna tupla dell'altro.
- Prodotto cartesiano degli operandi: ciascuna delle tuple di un operando è combinabile con tutte le tuple dell'altro.
Può verificarsi se le due relazioni non hanno attributi in comune.

Se ciascuna tupla di ciascun operando contribuisce ad almeno una tupla del risultato il join si dice **completo**.

Se per alcune tuple non è verificata la corrispondenza e quindi non contribuiscono al risultato, le tuple si dicono **dangling**.

8.3.1 Teorema 1

Se $X_1 \cap X_2$ è vuoto il join naturale equivale al prodotto cartesiano fra le relazioni.

8.3.1.1 Dimostrazione

Le due relazioni non hanno attributi comuni $\Rightarrow$ ogni tupla della prima relazione può "concatenarsi" con ogni tupla della seconda, soddisfacendo la definizione di join.

8.3.2 Teorema 2

Se le due relazioni hanno lo stesso schema ($X_1 = X_2$) il join naturale equivale all'intersezione fra le relazioni.

8.3.2.1 Dimostrazione

Ogni tupla può "concatenarsi" solo con una tupla uguale $\Rightarrow$ è presente nel risultato finale se e solo se è presente in entrambe le relazioni.

NB La concatenazione genera la tupla stessa.

8.3.3 Esempio join completo

Impiegato	Reparto	Reparto	Capo
Rossi	A	A	Mori
Neri	B	B	Bruni
Bianchi	B		

Impiegato	Reparto	Capo
Rossi	A	Mori
Neri	B	Bruni
Bianchi	B	Bruni

Ogni tupla contribuisce al risultato: il join è **completo**

8.3.4 Esempio join non completo

Impiegato	Reparto	Reparto	Capo
Rossi	A	B	Mori
Neri	B	C	Bruni
Bianchi	B		

Impiegato	Reparto	Capo
Neri	B	Mori
Bianchi	B	Mori

8.3.5 Esempio join vuoto

Impiegato	Reparto	Reparto	Capo
Rossi	A	D	Mori
Neri	B	C	Bruni
Bianchi	B		

Impiegato	Reparto	Capo

8.3.6 Proprietà

- $r_1 \Join r_2$ contiene un numero di tuple compreso fra zero (risultato = insieme vuoto) e il prodotto di $|r_1|$ e $|r_2|$ (risultato = prodotto cartesiano)
- Se $r_1 \Join r_2$ è completo allora contiene un numero di tuple pari almeno al massimo fra $|r_1|$ e $|r_2|$ (nessuna tupla resta esclusa, quindi tutte le tuple della relazione di cardinalità maggiore partecipano sicuramente al join)
- Se $X_1 \cap X_2$ contiene una chiave per r_2 , allora $r_1(X_1) \Join r_2(X_2)$ contiene al più $|r_1|$ tuple (ogni tupla di r_1 si associa al più a una tupla di r_2)
- Se il join coinvolge una chiave di r_2 e un vincolo di integrità referenziale da r_1 a r_2 , allora il numero di tuple è pari a $|r_1|$ (ogni tupla di r_1 si associa ad una e una sola tupla di r_2)
- Il join è commutativo: $r_1 \Join r_2 = r_2 \Join r_1$
- Il join è associativo: $(r_1 \Join r_2) \Join r_3 = r_1 \Join (r_2 \Join r_3)$
Quindi sequenze di join possono essere scritte senza parentesi

8.3.6.1 Theta-join e Equi-join

Per correlare attributi con nome diverso (se cioè $X_1 \cap X_2$ è vuoto) è possibile fare il **theta-join**, definito come un prodotto cartesiano seguito da una selezione:

$$r_1 \Join_F r_2 = \sigma_F(r_1 \Join r_2)$$

dove F è la condizione di selezione.

Se F è una condizione di uguaglianza fra un attributo della prima relazione e uno della seconda, allora siamo in presenza di un **equi-join**.

È il tipo di join più comune.

Sono importanti formalmente:

- Il join naturale è basato sui **nomi** degli attributi
- Equi-join e theta-join sono basati sui **valori**

8.3.6.2 Theta-join

Il theta-join, espresso come prodotto cartesiano seguito da una selezione, è il tipo di join operativamente più generale.

Infatti:

- Consente di associare (concatenare) tuple attraverso la corrispondenza fra attributi arbitrari e non necessariamente omonimi
- Consente di basare l'associazione su una arbitraria condizione e non solo su una condizione di uguaglianza

Impiegati		Reparti	
Impiegato	Reparto	Codice	Capo
Rossi	A	A	Mori
Neri	B	B	Bruni
Bianchi	B		

Impiegati		Reparti	
Impiegato	Reparto	Codice	Capo
Rossi	A	A	Mori
Neri	B	B	Bruni
Bianchi	B	B	Bruni

8.3.6.3 Equi-join

Impiegati		Reparti	
Impiegato	Reparto	Codice	Capo

Se uso ridenominazione + join naturale:

$$Impiegati \Join (REN_{Reparto \leftarrow Codice}(Reparti))$$

Il risultato è definito sullo schema:

Impiegato	Reparto	Capo
-----------	---------	------

Gli attributi resi omonimi dalla ridenominazione compaiono una sola volta.

Con un equi-join lo schema comprende i 4 attributi: quindi devo eseguire anche una proiezione per eliminare uno degli attributi "comuni" se voglio ottenere lo stesso risultato:

$$PROJ_{Impiegato, Reparto, Capo}(SEL_{Reparto=Codice}(Impiegati \Join Reparti))$$

8.3.7 Join Esterni

Il join naturale e il theta-join non includono nel risultato le tuple in cui non c'è corrispondenza fra gli attributi legati dal join (**tuple dangling**).

Questo fa sì che il risultato non contenga tutta l'informazione contenuta nelle relazioni di partenza.

Si definiscono altri tipi di join, detti **join esterni**, in cui vengono considerate anche le tuple dangling, utilizzando valori nulli dove non vi sia corrispondenza.

8.3.7.1 Join Sinistro

$$r_1 \text{ JOIN}_{LEFT} r_2$$

Contribuiscono tutte le tuple di r_1 , eventualmente estese con valori nulli relativamente agli attributi di r_2 se non c'è corrispondenza con tuple di r_2 .

Impiegati		Reparti	
Impiegato	Reparto	Reparto	Capo
Rossi	A	B	Mori
Neri	B	C	Bruni
Bianchi	B		

Impiegati JOIN_{LEFT} Reparti		
Impiegato	Reparto	Capo
Neri	B	Mori
Bianchi	B	Mori
Rossi	A	NULL

8.3.7.2 Join Destro

$$r_1 \text{ JOIN}_{RIGHT} r_2$$

Contribuiscono tutte le tuple di r_2 , eventualmente estese con valori nulli relativamente agli attributi di r_1 se non c'è corrispondenza con tuple di r_1 .

Impiegati		Reparti	
Impiegato	Reparto	Reparto	Capo
Rossi	A	B	Mori
Neri	B	C	Bruni
Bianchi	B		

Impiegati JOIN_{RIGHT} Reparti		
Impiegato	Reparto	Capo
Neri	B	Mori
Bianchi	B	Mori
NULL	C	Bruni

8.3.7.3 Join Completo o Full-Join

$$r_1 \text{ JOIN}_{FULL} r_2$$

Contribuiscono tutte le tuple di r_1 e r_2 , eventualmente estese con valori nulli relativamente agli attributi dell'altra relazione se non c'è corrispondenza con tuple dell'altra relazione.

Impiegati		Reparti	
Impiegato	Reparto	Reparto	Capo
Rossi	A	B	Mori
Neri	B	C	Bruni
Bianchi	B		

Impiegati JOIN_{FULL} Reparti		
Impiegato	Reparto	Capo
Neri	B	Mori
Bianchi	B	Mori
Rossi	A	NULL
NULL	C	Bruni

8.3.8 Join e Proiezioni

8.3.8.1 Problema: Perdite

Impiegato	Reparto	Reparto	Capo
Rossi	A	B	Mori
Neri	B	C	Bruni
Bianchi	B		

Impiegato	Reparto	Capo
Neri	B	Mori
Bianchi	B	Mori

Impiegato	Reparto	Reparto	Capo
Neri	B	B	Mori
Bianchi	B		

8.3.8.2 Problema: Non Reversibilità

Impiegato	Reparto	Capo
Neri	B	Mori
Bianchi	B	Bruni
Verdi	A	Bini

Impiegato	Reparto	Reparto	Capo
Neri	B	B	Mori
Bianchi	B	B	Bruni
Verdi	A	A	Bini

Impiegato	Reparto	Capo
Neri	B	Mori
Bianchi	B	Bruni
Neri	B	Bruni
Bianchi	B	Mori
Verdi	A	Bini

8.3.8.3 Proprietà

- Date due relazioni $R_1(X_1)$ e $R_2(X_2)$
 $\Pi_{X_1}(R_1 \Join R_2) \subseteq R_1$
 $\Pi_{X_2}(R_1 \Join R_2) \subseteq R_2$

Se proietto il risultato del join sullo schema di una delle due relazioni ottengo un sottoinsieme della relazione di partenza.

Infatti, se non c'è corrispondenza fra alcune tuple di R_1 e di R_2 , tali tuple sono dangling e vengono escluse dal join.

Se si tenta quindi di ricostruire le due relazioni originali si ha perdita di informazione.

- $R(X)$, $X = X_1 \cup X_2$
 $(\Pi_{X_1}(R)) \Join (\Pi_{X_2}(R)) \supseteq R$

Se scompongo lo schema di una relazione R in due sottoschemi X_1 e X_2 , R è contenuta nella relazione che si ottiene effettuando il join fra le proiezioni di R su di essi.

Quindi, se cerco di ricostruire le relazioni di partenza riproiettando il risultato del join sugli schemi di tali relazioni, ottengo due relazioni che comprendono le tuple delle relazioni di partenza più altre tuple "spurie".

9 Interrogazioni (Query)

Un'interrogazione è una funzione $E(r)$ che applicata ad istanze r di una base di dati produce una relazione su un dato insieme di attributi X .

Le interrogazioni su uno schema di base di dati R in algebra relazionale sono espressioni i cui atomi (le variabili) sono relazioni in R .

9.1 Esempio

Impiegati			
Matricola	Nome	Età	Stipendio
7309	Rossi	34	45
5998	Bianchi	37	38
9553	Neri	42	35
5698	Bruni	43	42
4076	Mori	45	50
8123	Lupi	46	60

Supervisione	
Impiegato	Capo
7309	5698
5998	5698
9553	4076
5698	4076
4076	8123

Trovare matricola, nome, età e stipendio degli impiegati che guadagnano più di 40:

$$SEL_{Stipendio > 40}(Impiegati)$$

Impiegati			
Matricola	Nome	Età	Stipendio
7309	Rossi	34	45
5698	Bruni	43	42
4076	Mori	45	50
8123	Lupi	46	60

Trovare matricola, nome ed età degli impiegati che guadagnano più di 40

$$PROJ_{Matricola, Nome, Età}(SEL_{Stipendio > 40}(Impiegati))$$

Impiegati		
Matricola	Nome	Età
7309	Rossi	34
5698	Bruni	43
4076	Mori	45
8123	Lupi	46

9.2 Procedimento Logico

Trovare le matricole dei capi degli impiegati che guadagnano più di 40:

$$PROJ_{Capo}(SEL_{Stipendio > 40}(Supervisione JOIN_{Impiegato=Matricola} Impiegati))$$

Operando secondo lo schema che abbiamo visto:

1. Creo una singola tabella in cui ogni tupla contiene (fra gli altri) gli attributi Capo, Impiegato e Stipendio con un join fra Supervisione e Impiegati
2. Seleziono le tuple con stipendio > 40
3. Proietto il risultato sull'attributo Capo

9.3 Algebra con valori nulli

Gli operatori logici vengono estesi con una logica a 3 valori: VERO (V), FALSO (F), SCONOSCIUTO (U)

Tabelle di verità operatori logici:

NOT		AND	V	U	F	OR	V	U	F
F	V	V	V	U	F	V	V	V	V
U	U	U	U	U	F	U	V	U	U
V	F	F	F	F	F	F	V	U	F

9.4 Equivalenze di espressioni

Due espressioni sono equivalenti se:

- $E_1 \equiv_R E_2$ se $E_1(r) = E_2(r)$ per ogni istanza r di R
Equivalenza dipendente dallo schema
- $E_1 \equiv E_2$ se $E_1 \equiv_R E_2$ per ogni schema R
Equivalenza assoluta

L'equivalenza è importante in quanto consente di scegliere, a parità di risultato, l'operazione meno costosa.

- Atomizzazione delle selezioni
 $\sigma_{F_1 \wedge F_2}(E) \equiv \sigma_{F_1}(\sigma_{F_2}(E))$
- Idempotenza delle proiezioni
 $\Pi_X(E) \equiv \Pi_X(\Pi_{XY}(E))$ qualunque sia Y
- Anticipazione della selezione rispetto al join
 $\sigma_F(E_1 \bowtie E_2) \equiv E_1 \bowtie (\sigma_F(E_2))$
- Anticipazione della proiezione rispetto al join:
 $\Pi_{X_1, Y_2}(E_1 \bowtie E_2) \equiv E_1 \bowtie \Pi_{Y_2}(E_2)$ **NB: $Y_1 \subseteq X_1, Y_2 \subseteq X_2$**
(E_1 definita su X_1 , E_2 definita su X_2)
(Vale solo se Y_2 contiene gli attributi coinvolti nella condizione di join)

Allora (combinando con idempotenza delle proiezioni):

$$\Pi_Y(E_1 \bowtie_F E_2) \equiv \Pi_Y(\Pi_{Y_1}(E_1) \bowtie_F \Pi_{Y_2}(E_2))$$

Dove Y_1 e Y_2 contengono, rispettivamente, gli attributi di X_1 e X_2 compresi in Y o coinvolti nella condizione F .

In pratica è possibile ignorare, in ciascuna relazione, gli attributi non compresi in Y e non coinvolti nel join.

- Inglobamento di una selezione in un prodotto cartesiano a formare un join (theta-join):
 $\sigma_F(E_1 \bowtie E_2) \equiv E_1 \bowtie_F E_2$
- Tutti gli operatori binari eccetto la differenza godono delle proprietà associativa e commutativa.
- Distributività della selezione rispetto all'unione:
 $\sigma_F(E_1 \cup E_2) \equiv \sigma_F(E_1) \cup \sigma_F(E_2)$
- Distributività della selezione rispetto alla differenza:
 $\sigma_F(E_1 - E_2) \equiv \sigma_F(E_1) - \sigma_F(E_2)$
- Distributività della proiezione rispetto all'unione:
 $\Pi_X(E_1 \cup E_2) \equiv \Pi_X(E_1) \cup \Pi_X(E_2)$
NB La proiezione NON è distributiva rispetto alla differenza
- Corrispondenze fra operatori insiemistici e selezioni complesse:
 $\sigma_{F_1 \vee F_2}(R) \equiv \sigma_{F_1}(R) \cup \sigma_{F_2}(R)$
 $\sigma_{F_1 \wedge F_2}(R) \equiv \sigma_{F_1}(R) \cap \sigma_{F_2}(R) \equiv \sigma_{F_1}(R) \Rightarrow \sigma_{F_2}(R)$
 $\sigma_{F_1 \wedge \neg F_2}(R) \equiv \sigma_{F_1}(R) - \sigma_{F_2}(R)$
- Proprietà distributiva del join rispetto all'unione:
 $E \Rightarrow (E_1 \cup E_2) \equiv (E \Rightarrow E_1) \cup (E \Rightarrow E_2)$

9.5 Viste (Relazioni Derivate)

- Rappresentazioni diverse per gli stessi dati (schema esterno):
 - **Relazioni di base**: contenuto autonomo; fisicamente e originariamente contenute nella base di dati
 - **Relazioni derivate**: relazioni il cui contenuto è funzione del contenuto di altre relazioni (definito per mezzo di interrogazioni)
 - **Relazioni Virtuali (Viste)**: Relazioni definite mediante funzioni o espressioni del linguaggio di interrogazione, non realmente presenti nella base di dati ma utilizzabili come se lo fossero. Devono essere ricalcolate tutte le volte.
 - **Viste materializzate**: Relazioni virtuali effettivamente inserite nella base di dati. Immediatamente disponibili ma critiche per il mantenimento dell'allineamento con le relazioni da cui derivano. Non sono supportate dai DBMS.

9.5.1 Vantaggi

- Permettono di mostrare a un utente le sole componenti della base di dati che interessano
- Espressioni molto complesse possono essere definite come viste e rese molto più leggibili
- Sicurezza: è possibile definire diritti di accesso anche per una sola vista (quindi per una particolare porzione della base di dati)
- In caso di ristrutturazione della base di dati, le “vecchie” relazioni possono essere di nuovo ricavate mediante viste, consentendo l'uso di applicazioni che fanno riferimento al vecchio schema

9.5.2 Esempio

Afferenza		Direzione	
Impiegato	Reparto	Reparto	Capo
Rossi	A	A	Mori
Neri	B	B	Bruni
Bianchi	B		

Una possibile vista:

$$Supervisione(Impiegato, Capo) = PROJ_{Impiegato, Capo}(Afferenza JOIN Direzione)$$

9.5.3 Interrogazioni sulle viste

Sono eseguite sostituendo alla vista la sua definizione:

$$SEL_{Capo='Leoni'}(Supervisione)$$

viene eseguita come

$$SEL_{Capo='Leoni'}(PROJ_{Impiegato, Capo}(Afferenza JOIN Direzione))$$

9.5.4 Selezione con condizione valutata su tuple diverse della stessa relazione

Trovare gli impiegati con lo stesso nome.

Data la relazione:

$$Impiegato(Nome, Cognome, MatricolaImpiegato)$$

Definisco una vista:

$$Impiegato1(Nome1, Cognome1, MatricolaImpiegato1)$$

=

$$REN_{Nome1, Cognome1, MatricolaImpiegato1 \leftarrow Nome, Cognome, MatricolaImpiegato}(Impiegato)$$

Interrogazione che ottiene il risultato voluto:

$$PROJ_{Nome, Cognome}(SEL_{Nome=Nome1}(Impiegato JOIN Impiegato1))$$

Prodotto Cartesiano

9.5.5 Viste come strumenti di programmazione

Trovare gli impiegati che hanno lo stesso capo di Rossi

- Senza vista: (I "..." significano che va a capo)
 $PROJ_{Impiegato}((Afferenza JOIN Direzione) JOIN REN_{ImpR, RepR \leftarrow Impiegato, Reparto} \dots (SEL_{Impiegato='Rossi'}(Afferenza JOIN Direzione)))$
- Con Vista:
 $PROJ_{Impiegato}((Supervisione) JOIN REN_{ImpR, RepR \leftarrow Impiegato, Reparto}(SEL_{Impiegato='Rossi'}(Supervisione)))$

NB In questo esempio, rispetto al precedente:

$$Supervisione(Impiegato, Reparto, Capo) = Afferenza JOIN Direzione$$

È senza proiezione, quindi contiene anche l'attributo Reparto!

9.5.6 Viste ed Aggiornamenti

Afferenza		Direzione	
Impiegato	Reparto	Reparto	Capo
Rossi	A	A	Mori
Neri	B	B	Bruni
Verdi	A	C	Bruni

Supervisione	
Impiegato	Capo
Rossi	Mori
Neri	Bruni
Verdi	Mori

Vogliamo inserire, nella vista, il fatto che Lupi ha come capo Bruni; oppure che Belli ha come capo Falchi; come facciamo?

- "Aggiornare una vista": modificare le relazioni di base in modo che la vista, "ricalcolata", rispecchi l'aggiornamento
- L'aggiornamento sulle relazioni di base corrispondente a quello sulla vista deve essere univoco
- In generale però non è univoco!
- Sulle viste sono ammissibili pochissimi aggiornamenti

10 Esercizi

- Trovare nome e stipendio dei capi degli impiegati che guadagnano più di 40
- Trovare gli impiegati che guadagnano più del proprio capo, mostrando matricola, nome e stipendio dell'impiegato e del capo
- Trovare le matricole dei capi i cui impiegati guadagnano **tutti** più di 40 (suggerimento: usare gli operatori insiemistici)